

Transcripción de operadores

Clara Rodríguez, Alejandro Hernández y Lucía Martínez

24 de septiembre de 2025

Resumen

Este documento está destinado para la transcripción de operadores C a su equivalente en lenguaje circom.

Guía de contenidos

1. Operadores Lógicos
2. Operadores de Comparación
3. Operadores Aritméticos
4. Acceso a Arrays
5. Expresiones condicionales
6. Operadores de desplazamiento

1. Operadores Lógicos

C	Circom	Comentario
<code>!a</code>	<code>1 - a</code>	Si a es condición, su negación se consigue con la diferencia entre 1 y ésta
<code>a && b</code>	<code>a * b</code>	Si a y b son verdaderos, el resultado de su producto será 1, de lo contrario será 0
<code>a b</code>	<code>1 - (1-a)*(1-b)</code> ó <code>a+b - (a*b)</code>	Si a , b o ambos son verdaderos, el resultado de su producto será 1, de lo contrario será 0

2. Operadores de Comparación

C	Circom	Comentario
<code>a == b</code>	<code>IsZero()(a - b)</code>	Si $a = b$, entonces se cumple ($=1$)
<code>a != b</code>	<code>1- IsZero()(a - b)</code>	Si $a \neq b$, entonces es falso ($=0$), por tanto se invierte con la resta
<code>a < b</code>	<code>LessThan(n)(a,b)</code>	Si $a < b$ es verdadero, entonces se cumple ($=1$)
<code>a ≤ b</code>	<code>LessThanEq(n)(a,b)</code>	Si $a \leq b$ es verdadero, entonces se cumple ($=1$)
<code>a > b</code>	<code>1- LessThanEq(n)(a,b)</code>	Si $a \leq b$ es falso ($=0$), entonces $a > b$
<code>a ≥ b</code>	<code>1- LessThan(n)(a,b)</code>	Si $a < b$ es falso ($=0$), entonces $a \geq b$

Nota: Las funciones `IsZero()` son circuitos que comparan si un valor es igual a 0.

Nota: Las funciones `LessThan(n)` y `LessThanEq(n)` son circuitos que comparan dos valores ($a < b$) de hasta n bits.

3. Operadores Aritméticos

Tanto en C como en Circom, los operadores suma (+), resta (-) y multiplicación (*) no difieren. En cambio, en operaciones como la división entera (/) y el módulo (%) si que encontramos diferencias y por ende es necesaria su transcripción.

La idea desde la que se parte para encontrar esta equivalencia es de la propiedad fundamental de la división entera, la cual establece:

$$a = b \cdot q + r$$

Operación división

Partiendo de la fórmula $a = b \cdot q + r$, podemos llegar al resultado de la operación a/b forzando a que se cumpla la fórmula pero despreciando el resto (ya que se trata de división entera).

El término "forzar" hace referencia a usar una variable en Circom, de forma que partiendo del dividendo y del divisor, el cociente solo pueda tener ese valor.

Ejemplo de código en Circom:

```
1      template IntegerDivision() {
2          signal input a; // Dividendo
3          signal input b; // Divisor
4          signal output c; // Cociente q
5
6          var q;
7          a == b * q;
8          c <= q;
9      }
10
11      component main = IntegerDivision();
```

Operación módulo

Partiendo de la fórmula $a = b \cdot q + r$, podemos llegar al resultado de la operación $a \% b$ con la variable r, la que representa el resto de una división.

$$a = b \cdot q + r \quad (1)$$

$$r = a - (b \cdot q) \quad (2)$$

$$r = a - (b \cdot (a/b)) \quad (3)$$

Nota: q representa el resultado de una división, por lo que podemos decir que $q = a/b$.

C	Circom	Comentario
a/b	<code>a == b * q</code>	Forzando q con una variable a que esa ecuación se cumpla
$a \% b$	<code>a - (b * (a/b))</code>	a/b hace referencia a la equivalencia que se obtiene en circom para poder realizar la operación %

4. Acceso a Arrays

La diferencia principal entre C y Circom en el acceso a arrays radica en que el tamaño de los arrays en Circom ha de ser fijo a la hora de compilar y ser siempre constante, mientras que en C puede ser estático y dinámico. Además, en Circom el valor de los arrays no puede ser reasignado dinámicamente.

Nota: En C, el tamaño de un array puede ser variable, ya sea por arrays de longitud variable o por el uso de memoria dinámica

1	<code>//Codigo en C</code>	1	<code>//Codigo circom</code>
2	<code>#include <stdio.h></code>	2	<code>template accesoArray(N) {</code>
3		3	<code>signal input lista[N]; //</code>
4	<code>int main() {</code>	4	<code>Tamanyo fijo, suponemos</code>
5			<code>iniciada</code>
6	<code>// Ejemplo de array con</code>		<code>signal output sumaAcumulada;</code>
	<code>tamanyo variable con</code>	5	
	<code>funcion f()</code>	6	
7	<code>int n = f();</code>	7	<code>var suma = 0;</code>
8	<code>int lista[n]; // Se decide</code>	8	<code>for(int i = 0; i < N; i++){</code>
	<code>en tiempo de</code>	9	<code>suma = suma + lista[i];</code>
	<code>ejecucion</code>	10	
9		11	<code>// NO se puede hacer lo</code>
10			<code>siguiente</code>
11	<code>// EJemplo de array de</code>	12	<code>// lista[i] <= 3;</code>
	<code>tamanyo variable con</code>	13	<code>}</code>
	<code>asignacion de memoria</code>	14	
	<code>dinamica</code>	15	<code>sumaAcumulada <= suma;</code>
12	<code>int *lista = malloc(x);</code>	16	<code>}</code>
13		17	
14	<code>//x tamanyo que se quiera</code>	18	<code>component main = accesoArray</code>
	<code>reservar en memoria</code>		<code>(10);</code>
	<code>dinamica</code>		
15	<code>}</code>		

Nota: En Circom no se puede fijar el tamaño de un array con una señal.

5. Expresiones condicionales

En C, al ser un lenguaje de ejecución dinámica, los condicionales pueden decidir el valor final de una variable en tiempo de ejecución.

En Circom, cada señal representa un “cable” del circuito, que solo puede recibir un valor. Por ello, no es posible reasignar un signal dentro de un condicional dinámico.

Para simular decisiones condicionales basadas en entradas (inputs), se deben usar expresiones aritméticas que implementen la lógica de selección, de manera que el valor de salida quede correctamente definido sin reasignar el signal.

1	<code>//Codigo en C</code>	1	<code>//Codigo circom</code>
2	<code>#include <stdio.h></code>	2	<code>template condicionales() {</code>
3		3	<code>signal input entrada;</code>
4	<code>int main(int argc, char *</code>	4	<code>signal output salida; //0</code>
	<code>argv[]) {</code>	5	<code>par, 1 impar</code>
5	<code>int entrada = argv[1];</code>	6	
6	<code>char mensaje[50];</code>	7	<code>var var_division;</code>
7		8	<code>entrada == 2 *</code>
8	<code>if(entrada % 2 == 0){</code>	9	<code>var_division;</code>
9	<code>sprintf(mensaje, "%d es</code>	10	<code>salida <= (entrada - (2*</code>
	<code>par\n", entrada);</code>	11	<code>var_division))</code>
10	<code>}</code>	12	<code>}</code>
11	<code>else {</code>	13	
12	<code>sprintf(mensaje, "%d es</code>	14	<code>component main =</code>
	<code>impar\n", entrada);</code>		<code>condicionales();</code>
13	<code>}</code>		
14			
15	<code>printf("s", mensaje);</code>		
16	<code>free(mensaje);</code>		
17			
18	<code>return 0;</code>		
19	<code>}</code>		

Con este ejemplo se ilustra como puede transformarse una expresión condicional que comprueba si un número es par o impar en una expresión aritmética que representa lo mismo y evita el reasignar la señal de salida más de una vez (algo no permitido).

6. Operadores de desplazamiento

Los operadores de desplazamiento en C funcionan de la siguiente manera:

Imaginemos que tenemos el número 2 que queremos desplazar 2 bits

- 1. Dado un número decimal 2, transformarlo a binario, que es 0010
- 2. Con el n (número de bits a desplazar), desplazamos los dos últimos bits
- 1000 (4 en decimal), es el número que se obtiene desplazando los bits

*Observación: Podemos observar que para obtener el número equivalente del desplazamiento de bits en decimal se consigue con $x * 2^n$ en el caso de desplazamiento a la derecha y $x / 2^n$ en el caso de desplazamiento a la izquierda.*

Podemos hacer uso de esta observación para llegar a la equivalencia de tal operación en Circom.

C	Circom
$a \ll b$	<pre> 1 2 //Codigo circom 3 template desplazamiento_izq(N) { 4 signal input numero_decimal; 5 signal output salida; 6 7 var potencia = 1; 8 9 for (var i = 0; i < N; i++) { 10 potencia = potencia * 2; 11 } 12 13 salida <== numero_decimal * potencia; 14 15 } 16 17 component main = desplazamiento_izq(N); </pre>
$a \gg b$	<pre> 1 2 //Codigo circom 3 template desplazamiento_dcha(N) { 4 signal input numero_decimal; 5 signal output salida; 6 7 var potencia = 1; 8 9 for (var i = 0; i < N; i++) { 10 potencia = potencia * 2; 11 } 12 13 var division; 14 15 numero_decimal === potencia * division; 16 17 salida <== division; 18 19 } 20 21 component main = desplazamiento_dcha(N); </pre>